King Fahd University of Petroleum and Minerals

ICS-344 - Information Security

Sections: 1 & 2

DVSA Vulnerability Discovery and Remediation

Term:	252
Student Name(s):	Husain Al Muallim[1], Ali Al Qatari[2], Abdulwahed Zuhair Adi[3]
Student ID(s):	202163730, 202279780, 202256380
AWS Region:	us-east-1
Date:	*Submission date*

DVSA Website URL:
`http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com`

Project Report

Contents

Project Overview

Scope

Summarize the project scope: deployment in a non-production AWS account, the 10 official lessons, remediation, and verification.

Deployment Summary

- DVSA Website URL: http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com
- AWS region: us-east-1
- CloudFormation stack: *Stack name*
- Main services used: *S3, API Gateway, Lambda, DynamoDB, SQS, Cognito, IAM, CloudWatch*

Architecture and Request Pipeline

Explain the intended high-level request flow from browser to API Gateway to Lambda to backend services and back.

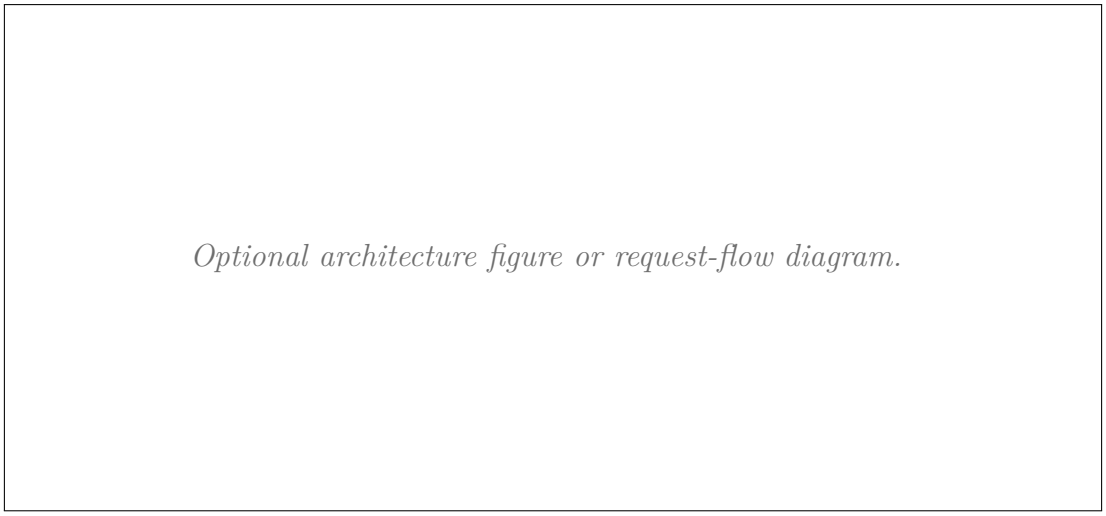

Figure 1: Architecture figure or request-flow diagram

Methodology and Evidence Sources

- *How the environment was validated before testing.*
- *What tools were used to reproduce and verify vulnerabilities.*
- *How evidence was captured and redacted.*
- *How fixes were verified after remediation.*

Chapter 1

Lesson 01: Event Injection

1.1 Part 1: Goal and Vulnerability Summary

The objective of this lesson is to successfully exploit an Event Injection flaw to achieve Remote Code Execution (RCE) on the backend cloud server, and subsequently remediate the vulnerability. The vulnerability lies within the `DVSA-ORDER-MANAGER` AWS Lambda function, which acts as the backend for the application's order system. When a user submits an HTTP request, the API Gateway passes the user's JSON payload directly to this Lambda function. Instead of safely parsing this text, the developer used an outdated and inherently insecure Node.js library called `node-serialize` to deserialize both the request body and the HTTP headers. Because the backend blindly trusts the incoming user input and processes it with this vulnerable library, an attacker can craft a malicious JSON payload containing executable code, bypassing normal application logic.

1.2 Part 2: Why This Works / Root Cause

The root cause is insecure deserialization. The `DVSA-ORDER-MANAGER` Lambda function processes incoming HTTP requests using `serialize.unserialize(event.body)`. Under the hood, the `node-serialize` package uses JavaScript's `eval()` function to execute code if it detects a specifically formatted string (starting with `_$$ND_FUNC$$_`). Because the backend does not validate or sanitize the incoming JSON payload before passing it to this function, the attacker's injected JavaScript payload is executed with the privileges of the Lambda function.

1.3 Part 3: Environment and Setup

- API endpoint:

  `https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage`

- AWS services involved: API Gateway, AWS Lambda, CloudWatch Logs.

- Relevant function, role, or log group: Lambda function `DVSA-ORDER-MANAGER` and its CloudWatch Log Group.
- User or test context: Unauthenticated external attacker.
- Tools used: Mac Terminal, `curl` for sending HTTP POST requests, and AWS Management Console (CloudWatch).

1.4 Part 4: Reproduction Steps

1. The baseline behavior of the `/order` endpoint is to accept a valid JSON payload containing order details and process it.
2. To trigger the vulnerability, a crafted JSON payload is sent to the `/order` API endpoint using `curl`.
3. The payload contains the malicious `_$$ND_FUNC$$_` tag, followed by a self-executing JavaScript function that writes a file to the Lambda's `/tmp` directory, reads it, and prints the contents to the console error log.
4. The following command was executed:

```
curl -X POST "https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/
    order" \
 -H "Content-Type: application/json" \
 -d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.
    writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
    hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"
    utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()", "cart
    -id":""}'
```

5. The terminal returns an `{"message": "Internal server error"}`, indicating the payload executed and crashed the backend function.

1.5 Part 5: Evidence and Proof

- CloudWatch logs were inspected to verify that the payload executed successfully.
- The logs clearly show the injected message "FILE READ SUCCESS: You are reading the contents of my hacked file!", proving that the remote code execution was successful.

Figure 1.1: Screenshot of Terminal Output showing the Internal Server Error

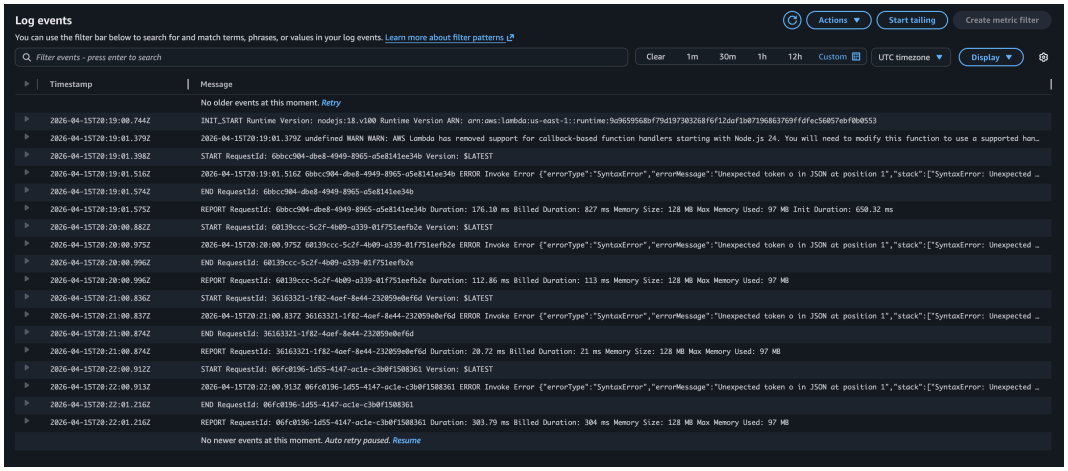

Figure 1.2: Screenshot of CloudWatch Log showing the "FILE READ SUCCESS" message

1.6 Part 6: Fix Strategy / Probable Mitigation

- Fix location: Backend Lambda `DVSA-ORDER-MANAGER`, file `order-manager.js`.
- Security property restored: Safe input handling and prevention of Remote Code Execution.
- Why this addresses root cause: The mitigation strategy requires removing the insecure deserialization mechanism entirely. The `node-serialize` library must be removed from the code. All incoming JSON data (both the event body and headers) must be parsed using the native, safe `JSON.parse()` method, which only interprets data as text/objects and never executes it as code.

1.7 Part 7: Code / Config Changes

The `order-manager.js` file in the `DVSA-ORDER-MANAGER` Lambda function was updated to replace the insecure library.

The following lines were modified:

- **Line 1:** `// const serialize = require('node-serialize');` (Commented out)
- **Line 10:** `var req = JSON.parse(event.body);` (Replaced unserialize)
- **Line 11:** `var headers = JSON.parse(event.headers);` (Replaced unserialize)

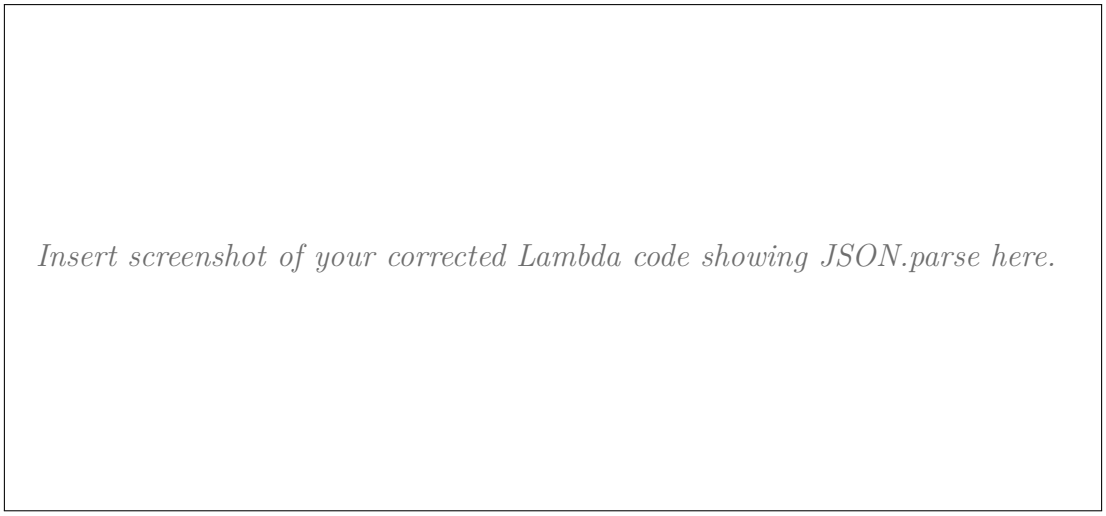

Figure 1.3: Corrected Lambda code for Lesson 01

1. After deploying the updated Lambda function, the exact same malicious `curl` command from Part 4 was executed in the terminal.
2. The terminal again returned an `{"message": "Internal server error"}` (due to a missing authentication header failing later in the safe code).
3. The CloudWatch logs for the `DVSA-ORDER-MANAGER` were reviewed again. The attack failed to execute, and the `FILE READ SUCCESS` message was no longer present in the logs, confirming the vulnerability is patched.

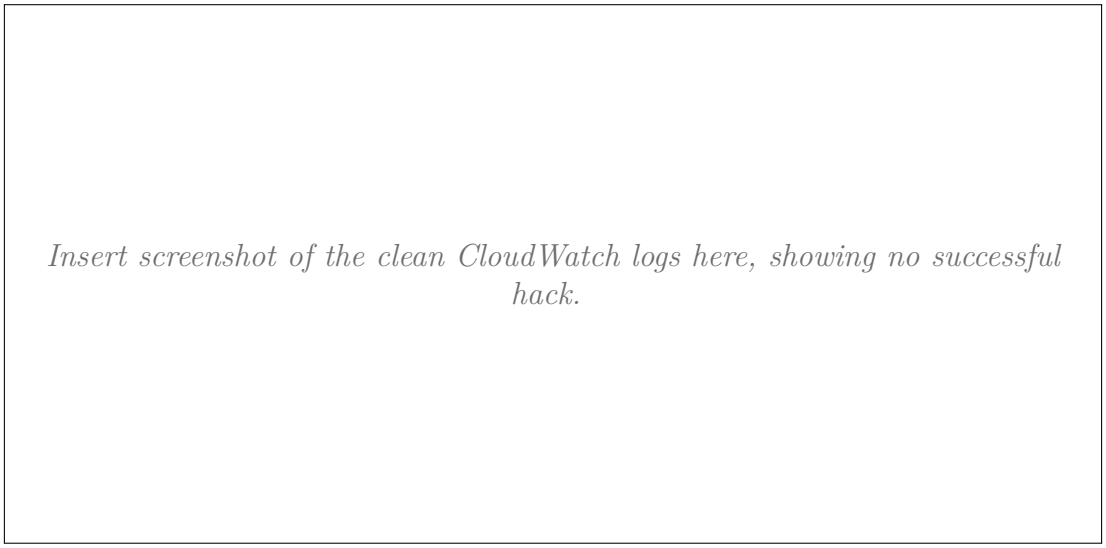

Figure 1.4: Post-fix CloudWatch logs for Lesson 01

Table A: Operational Analysis

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The backend should safely parse JSON data representing an order without executing arbitrary code contained within the payload.	Source code analysis of `order-manager.js` and normal application workflow.	The API accepts standard JSON payloads and processes order updates.	CloudWatch logs display output from custom JavaScript injected by the attacker.

Table B: Security Analysis

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The application evaluated user input as code rather than pure data.	Unsafe input handling / Insecure Deserialization.	Removed the `node-serialize` module and replaced `serialize.unserialize()` calls with `JSON.parse()`.	Re-running the exploit payload no longer prints the injected message to the CloudWatch logs.	N/A

1.8 Part 10: Takeaway / Lessons Learned

The primary takeaway is that user input should never be trusted, and development environments must never evaluate or execute code passed via external APIs. When handling serialized data like JSON, native and strictly typed parsers (like `JSON.parse()`) must be used instead of third-party libraries that rely on dynamic evaluation (like `eval()`). Furthermore, auditing dependencies for known vulnerabilities is critical for maintaining serverless security.

Optional Bonus Findings

Use this section only if you discover valid additional findings beyond the 10 official lessons. Repeat the same reporting discipline if you include bonus work.

Conclusion

Summarize the project outcomes, the most important remediation themes, and the main security lessons learned from DVSA.